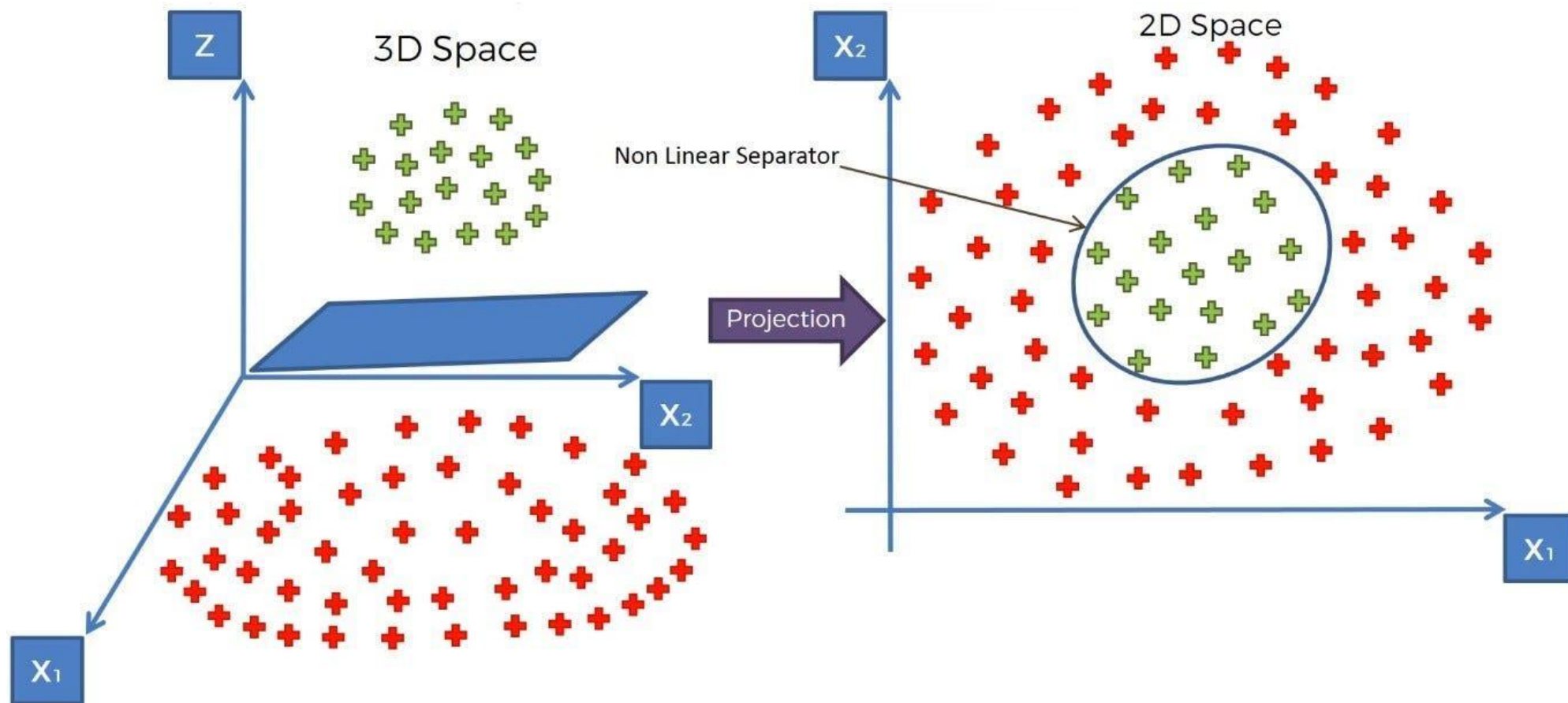


Sección 9: Support Vector Machine (SVM)



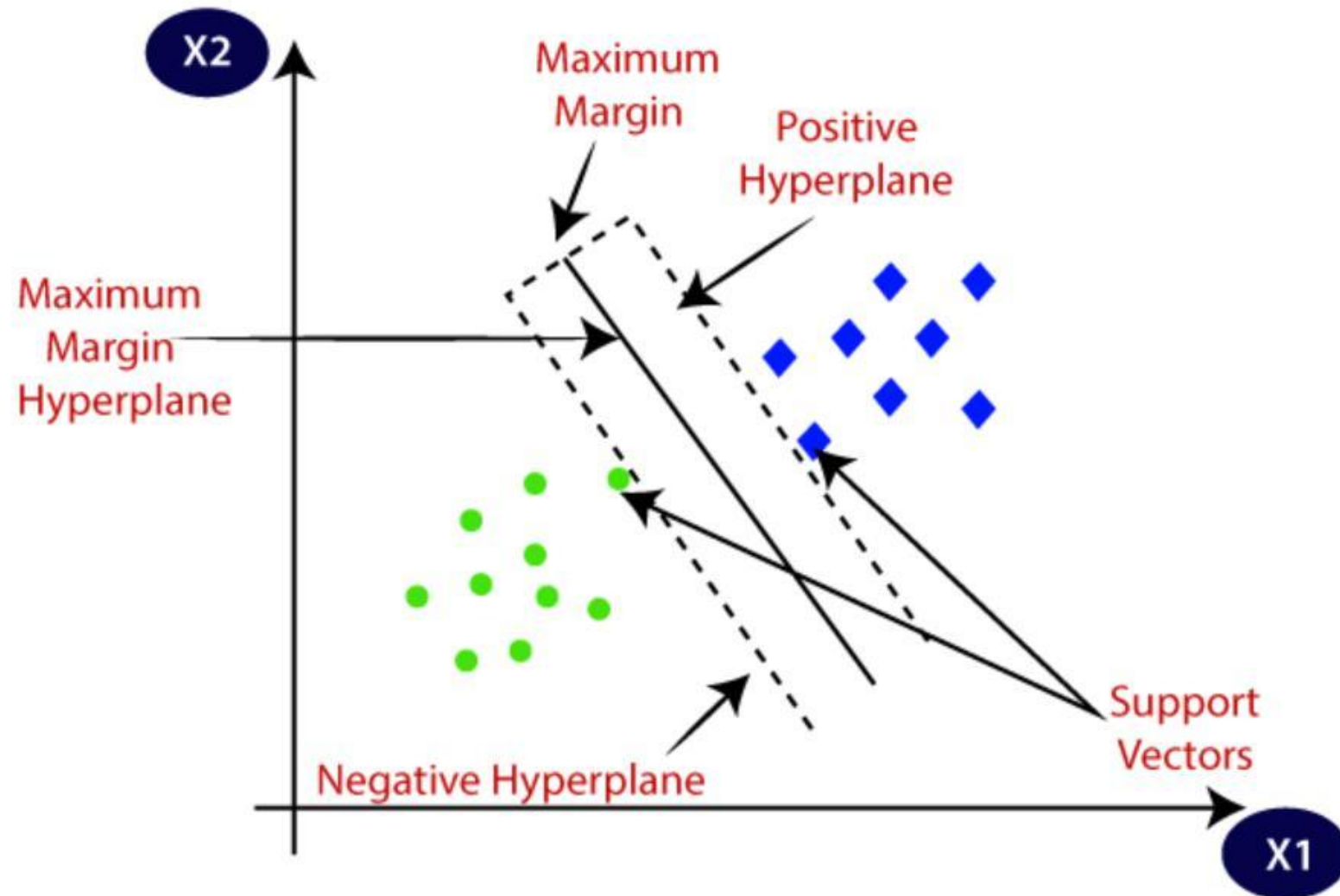
SVM: Introducción

- SVM es un algoritmo de aprendizaje supervisado capaz de realizar tanto regresión como clasificación (lineal y no lineal).
- Destaca con conjuntos de datos pequeños y complejos, a diferencia de muchas otras técnicas.
- SVM busca maximizar el margen entre clases, lo que permite una mejor generalización para nuevos datos, un concepto denominado "clasificación de amplio margen" ("large margin classification").
- SVM es capaz de construir tanto modelos lineales como no lineales. Esto le permite realizar predicciones en conjuntos de datos linealmente separables y no linealmente separables.

Bibliografía SVM

- [¿Qué es Support Vector Machine? | IBM](#)
- [2. Support Vector Machines \(SVM\). — Trabajando con scikit learn](#)
- [9 Modelo de Support Vector Machine \(SVM\) | Una introducción a los modelos de Clasificación empleando R](#)
- [Tutorial de SVM Scikit-learn con Python \(máquinas de vectores de soporte\) | DataCamp](#)
- [\(2224\) Aprende SVM \(Support Vector Machines\) con Python | Machine Learning 101 – YouTube](#)
- [Las matemáticas detrás de SVM](#)

SVM en conjuntos de datos linealmente separables



Dos categorías principales de algoritmos SVM para conjuntos linealmente separables

- **Hard Margin Classification:** Se aplica cuando el conjunto de datos es perfectamente linealmente separable y no hay ejemplos que se superpongan entre las clases. El objetivo es encontrar un límite de decisión que separe las clases sin errores y con el mayor margen posible.
- **Soft Margin Classification:** Se utiliza cuando el conjunto de datos no es perfectamente linealmente separable, es decir, hay algunos ejemplos que se superponen o están dentro del margen de la clase opuesta. Permite cierta tolerancia a errores de clasificación para encontrar un límite de decisión que aún maximice el margen, pero que también maneje los valores atípicos (“outliers”) o el ruido en los datos.

SVM: Hard Margin Classification

- Se centra en el método de SVM para construir un límite de decisión maximizando el margen entre dicho límite y los puntos de datos de entrenamiento más cercanos (vectores de soporte).
- Este método se limita a conjuntos de datos linealmente separables y es muy sensible a los valores atípicos (anómalos), lo que puede generar predicciones deficientes.

SVM: Construcción del modelo lineal – Función de coste

- La función de coste de las SVM, conocida como “pérdida de bisagra” (hinge loss), busca maximizar los márgenes.
- Para ejemplos de la clase positiva ($y=1$), el coste es 0 cuando $\theta \geq 1$.
- Para ejemplos de la clase negativa ($y=0$), el coste es 0 cuando $\theta \leq -1$.

SVM: Construcción del modelo lineal –Función hipótesis

- SVM ajusta iterativamente los parámetros para lograr un error cero, asegurando que las predicciones se sitúen muy por encima del límite de decisión y centrándose en los “vectores de soporte” más cercanos a éste.
- Los "support vectors" son los ejemplos de entrenamiento más cercanos al límite de decisión y son los puntos de referencia que el algoritmo utiliza para modificar el límite de decisión, alejándolo de ellos para maximizar esos márgenes.

SVM: Soft Margin Classification

- El objetivo de la clasificación de margen suave (soft margin classification) es mantener un balance adecuado entre mantener el límite de decisión lo más alejado posible de los ejemplos de entrenamiento, es decir, maximizar esos márgenes o esa distancia, y disminuir la sensibilidad a los ejemplos anómalos.
- Esto significa que, si hay un ejemplo anómalo en el conjunto de datos de entrenamiento, se sigue construyendo un límite de decisión que maximice la distancia o los márgenes entre el resto de ejemplos del conjunto de datos de entrenamiento, aunque haga una mala predicción para ese ejemplo anómalo en concreto.

SVM: Soft Margin Classification

- En la clasificación de margen suave de SVM, el hiperparámetro 'C' controla la flexibilidad con la que se construye el límite de decisión. No existen dos tipos de algoritmos, uno para hard margin classification y otro para soft margin classification, sino que la flexibilidad del modelo se regula mediante este hiperparámetro 'C'. Intuitivamente, 'C' controla la regularización aplicada al modelo o al algoritmo SVM, quitando peso a algunos ejemplos del conjunto de datos de entrenamiento. Esto significa que modifica la función hipótesis para dar menos influencia a ejemplos anómalos en la construcción del límite de decisión, haciendo que no sean los únicos vectores de soporte.

SVM: Hard vs Soft Margin Classification

- Scikit-learn siempre usa soft margin, pero podemos hacerlo tan estricto para que se comporte como hard margin.
- `model = SVC(C=1e6, kernel='linear')` (hard margin)
- $\min_{\theta} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \xi_i$ el hiperparámetro C se aplica sólo al error, y no a los parámetros de la función hipótesis, porque las anomalías generan errores muy grandes.
- $C = 0.1$ # flexible
- $C = 1$ # balanceado
- $C = 1000$ # casi hard margin
- Se requiere un dataset relativamente grande para observar lo anterior

SVM: Kernels – Regresión Polinómica para conjuntos no linealmente separables

- Aquí se explica cómo usar SVM para datos no linealmente separables. Introduce el concepto de kernels, específicamente el kernel gaussiano, y luego detalla la regresión polinómica como un método más sencillo para crear límites de decisión no lineales.

SVM: Kernels – Regresión Polinómica

- La regresión polinómica implica agregar características polinómicas (p. ej., x^2 , x^3) basadas en las características ya existentes, $x_1 \dots$ para crear modelos no lineales.
- Si bien es ilustrativa, la regresión polinómica tiene inconvenientes: puede requerir polinomios de alto grado, lo que conlleva ineficiencia computacional y dificultad para seleccionar las características óptimas.
- Estas limitaciones la hacen menos práctica para aplicaciones del mundo real en comparación con los métodos basados en kernels.

SVM: Kernels – Gaussian Kernel

- El algoritmo Kernel Gaussiano para SVM, se centra en cómo generar nuevas características para manejar datos no linealmente separables.
- Utiliza puntos de referencia para calcular medidas de similitud, transformando los datos a una dimensión superior.
- Esto permite a las SVM crear límites de decisión no lineales, clasificando eficazmente conjuntos de datos complejos.
- Una estrategia común consiste en establecer un punto de referencia para cada ejemplo de entrenamiento, creando una nueva característica para cada uno.

SVM: Kernels – Gaussian Kernel

- Las nuevas características se crean a partir de un conjunto de puntos de referencia (landmarks) y la distancia entre las características existentes y estos landmarks.
- Si un ejemplo está muy cerca de un landmark, la distancia euclidiana será cercana a cero, y el valor de la nueva característica (la medida de similitud) será próximo a uno. Por el contrario, si un ejemplo está alejado de un landmark, la distancia euclidiana será un valor grande, y el valor de la nueva característica será aproximadamente cero.
- De esta manera, cada landmark contribuye a la creación de una nueva característica que indica la proximidad de los ejemplos a ese punto de referencia específico.

SVM: Kernels – Gaussian Kernel

- Estas nuevas características reemplazan a las características originales en la función hipótesis del SVM.
- La función hipótesis resultante, que ahora utiliza estas nuevas características, es capaz de generar un límite de decisión no lineal, lo que permite separar conjuntos de datos que no son linealmente separables.

Sección 10: Árboles de decisión y Random Forest (Foresta Aleatoria)

Árboles de decisión y Random Forest

- Estos algoritmos son eficaces incluso con un preprocesamiento de datos mínimo.
- Ofrecen un rendimiento aceptable y son menos complejos de comprender que algoritmos como las Máquinas de Vectores de Soporte.
- Estos algoritmos son ampliamente utilizados y **proporcionan muy buenos resultados, incluso con conjuntos de datos que no han sido escalados ni normalizados.**

Árboles de decisión: Introducción

- Los árboles de decisión construyen límites de decisión no lineales mediante una serie de reglas "si-entonces-si no".
- Pueden predecir valores tanto continuos (regresión) como discretos (clasificación).
- Se ilustra su funcionamiento con un ejemplo de clasificación multiclase, mostrando cómo los datos se dividen recursivamente según las características de entrada (X_1 , X_2) para crear un modelo.
- Son clasificadores no lineales, lo que les permite clasificar conjuntos de datos que no son linealmente separables, construyendo modelos o límites de decisión no lineales, a diferencia de la regresión logística o el algoritmo SVM que construyen límites con líneas o planos rectos.

Árboles de decisión: Algoritmo

- Inicialmente, se tiene un conjunto de datos de entrenamiento con 40 ejemplos, distribuidos en tres clases: 12 ejemplos de la clase 0 (verde), 12 ejemplos de la clase 1 (rojo) y 16 ejemplos de la clase 2 (azul). El objetivo es construir un modelo que separe estas tres clases.
- El proceso intuitivo comienza con la identificación de un punto de corte en una de las características de entrada. Por ejemplo, se puede establecer un límite en $X_1 = 3$. Esto divide el conjunto de datos original en dos subconjuntos:
 1. Subconjunto 1 ($X_1 \leq 3$): Este subconjunto contiene todos los ejemplos cuya característica X_1 es menor o igual a 3. En el ejemplo, este subconjunto estaría compuesto por 12 ejemplos de la clase 0 (verde) y 0 ejemplos de las clases 1 y 2. Dado que todos los ejemplos en este subconjunto pertenecen a la misma clase (clase 0), no es necesario subdividirlo más.
 2. Subconjunto 2 ($X_1 > 3$): Este subconjunto contiene todos los ejemplos cuya característica X_1 es mayor que 3. En el ejemplo, este subconjunto estaría compuesto por 0 de la clase 0, 12 de la clase 1 (rojo) y 16 de la clase 2 (azul). Como este subconjunto aún contiene ejemplos de dos clases diferentes, se necesita una subdivisión adicional.

Árboles de decisión: Algoritmo

- Para subdividir el Subconjunto 2, se busca otro punto de corte, esta vez en la característica X_2 . Por ejemplo, se puede establecer un límite en $X_2 = 4$. Esto divide el Subconjunto 2 en dos nuevos subconjuntos:
 1. Subconjunto 2a ($X_2 \leq 4$): Este subconjunto contiene los ejemplos del Subconjunto 2 cuya característica X_2 es menor o igual a 4. En el ejemplo, este subconjunto estaría compuesto por 16 ejemplos de la clase 2 (azul) y 0 ejemplos de las clases 0 y 1. Dado que todos los ejemplos en este subconjunto pertenecen a la misma clase (clase 2), no es necesario subdividirlo más.
 2. Subconjunto 2b ($X_2 > 4$): Este subconjunto contiene los ejemplos del Subconjunto 2 cuya característica X_2 es mayor que 4. En el ejemplo, este subconjunto estaría compuesto por 12 ejemplos de la clase 1 (rojo) y 0 ejemplos de las clases 0 y 2. Dado que todos los ejemplos en este subconjunto pertenecen a la misma clase (clase 1), no es necesario subdividirlo más.

Árboles de decisión: Entrenamiento del algoritmo

- Comenzar con datos etiquetados y analizar las variables de entrada para encontrar el mejor punto de división.
- Evaluar iterativamente los posibles valores de división para cada variable utilizando métricas como la impureza de Gini.
- Aplicar este proceso recursivamente a subconjuntos hasta que los nodos sean puros (contengan solo una clase) o se cumpla un criterio de parada temprana (*early stopping*) para evitar el sobreajuste.

Árboles de decisión: Algoritmo

- Este proceso de división recursiva crea un "árbol" de decisiones, donde cada nodo representa una regla de corte basada en una característica de entrada, y las hojas del árbol representan las clases finales. Los límites de decisión resultantes no son líneas o planos rectos, sino una serie de segmentos que se ajustan a la distribución de los datos, lo que permite a los árboles de decisión clasificar conjuntos de datos que no son linealmente separables.
- Cuando se presenta un nuevo ejemplo para predecir su clase, se evalúa a través de las reglas del árbol. Por ejemplo, si un nuevo ejemplo tiene $X1 = 4$ y $X2 = 3$:
 1. Se evalúa la primera regla: ¿ $X1 \leq 3$? No, porque $X1$ es 4.
 2. Se pasa al siguiente nodo (Subconjunto 2).
 3. Se evalúa la siguiente regla: ¿ $X2 \leq 4$? Sí, porque $X2$ es 3.
 4. Se llega a un nodo hoja que corresponde a la clase 2.

De esta manera, el árbol de decisión clasifica el nuevo ejemplo como perteneciente a la clase 2.

Árboles de decisión: Gini Impurity

- Gini impurity (o índice de Gini) es una medida que indica qué tan “mezcladas” están las clases dentro de un nodo.
- ¿Qué significa intuitivamente?
 - Impureza baja (cerca de 0): el nodo contiene casi todos los datos de una sola clase → está “ordenado”.
 - Impureza alta (cerca de 0.5 en binario): hay mezcla de clases → está “desordenado”.
- **Definición matemática**
- $$\text{Gini} = 1 - \sum_{i=1}^K p_i^2$$
- Donde:
- K: número de clases
- p_i : proporción de la clase i en el nodo

Árboles de decisión: Gini Impurity

- En un nodo, el algoritmo (por ejemplo CART):
- **Prueba muchas particiones posibles**, por ejemplo:
 - distintos atributos
 - distintos umbrales (si es numérico)
- Para cada partición calcula el **Gini ponderado** de los nodos hijos:
- Luego:
 - **compara todos los resultados**
 - **elige la partición con menor Gini**

Árboles de decisión: Gini Impurity

Si una partición da un **Gini cercano a 1**:

- **No se selecciona** (porque es mala, muy impura)
- **Pero tampoco se “desecha” el nodo completo**

Simplemente:

- se prueba otra partición
- se sigue evaluando hasta encontrar la mejor

Se detiene cuando:

- el nodo es puro ($\text{Gini} = 0$)
- no hay mejora significativa
- se alcanza profundidad máxima
- hay pocas muestras

Árboles de decisión: Gini Impurity

- Ejemplo simple (clasificación binaria)

- Supongamos un nodo con:

80% clase A

20% clase B

- Entonces:

$$\text{Gini} = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 0.32$$

- Interpretación: hay cierta mezcla, pero domina la clase A.

Árboles de decisión: Gini Impurity

En otras palabras:

- Se toma el primer nodo y selecciona una característica, se escoge un valor inicial, se hace la partición, y se calcula el coeficiente Gini para los dos nodos hijos, se selecciona un siguiente valor para la característica escogida y se calcula el coeficiente Gini para los dos nodos hijos, y así sucesivamente hasta el último valor escogido para dicha característica.
- Ahora, se toma de nuevo el primer nodo, y se selecciona la segunda característica y se hace todo el proceso anterior. Y así sucesivamente con el resto de las características.
- Considerano los coeficientes Gini generados hasta ahora, se escoge la variable inicial que presente el Gini más pequeño, y se pasa al siguiente nodo en profundidad, y se hace todo el proceso anterior para escoger la siguiente variable de corte

Árboles de decisión: Gini Impurity

- **¿Para qué sirve en árboles de decisión?**

El Gini impurity se usa para **decidir cómo dividir los datos en cada nodo** del árbol.

- **Objetivo del algoritmo:**

Buscar divisiones que:

- Disminuyan la impureza
- Generen nodos lo más “puros” posible

Es decir, el árbol intenta **separar bien las clases**.

Árboles de decisión: Gini Impurity

- ¿Cómo se aplica en la práctica?

En cada nodo:

1. Se prueban diferentes posibles divisiones (por ejemplo: edad < 30 , ingreso > 500 , etc.)
 2. Para cada división: Se calcula el Gini de los nodos hijos
 3. Se elige la división que:
 - Minimiza la impureza total
- En resumen: El Gini mide cuán confundido está el nodo. El árbol intenta hacer preguntas que reduzcan esa confusión lo más posible

Árboles de decisión: Gini Impurity

- Comparación rápida con otros criterios
 - **Gini impurity** → más rápido de calcular, muy usado (por ejemplo en RandomForest)
 - **Entropía** (Information Gain) → más teórico (basado en información)
- Ambos buscan lo mismo: **nodos más puros**

Árboles de decisión: ¿El árbol crece hasta pureza total?

- En teoría: sí.
- En la práctica: no siempre.

1. Crecimiento “completo” (sin restricciones)

- Si no impones límites, el algoritmo sigue dividiendo hasta que:
 - Cada nodo tiene **una sola clase** (Gini = 0)
 - O no hay más formas de dividir los datos
- Esto produce un árbol **muy profundo y muy específico**
- Problema:
Esto suele llevar a **overfitting (sobreajuste)**
→ El modelo memoriza los datos de entrenamiento, pero generaliza mal

Árboles de decisión: ¿El árbol crece hasta pureza total?

2. Detención temprana (early stopping) (lo más usado)

En la práctica, se detiene antes usando criterios de parada:

Algunos de los más importantes:

- `max_depth` → profundidad máxima del árbol
- `min_samples_split` → mínimo de datos para dividir un nodo
- `min_samples_leaf` → mínimo de datos en una hoja
- `max_leaf_nodes` → número máximo de hojas
- `min_impurity_decrease` → mínima reducción del Gini para dividir

Esto se llama pre-pruning (poda temprana)

Árboles de decisión: ¿El árbol crece hasta pureza total?

- 3. Poda posterior (post-pruning)

Otra estrategia es:

1. Dejar crecer el árbol bastante
2. Luego recortar ramas que no aportan valor

Esto se llama post-pruning

En scikit-learn, por ejemplo, se usa:

`ccp_alpha` → controla cuánto se poda el árbol

En resumen:

El árbol puede crecer hasta clasificar perfectamente los datos de entrenamiento, pero si lo hace demasiado, aprende 'de memoria' en vez de aprender patrones

- Árbol muy profundo → muchas hojas puras pero poco generalizable
- Árbol más simple → menos perfecto, pero más robusto

Árboles de decisión: Regresión y Función de Coste

- Los árboles de decisión para la regresión utilizan una función de coste de Error Cuadrático Medio (ECM) en lugar de medidas de impureza.
- Las predicciones en los nodos hoja son el promedio de los valores objetivo de los ejemplos de entrenamiento en ese nodo.
- El algoritmo selecciona las divisiones que minimizan este ECM.
- Para las tareas de regresión, los árboles de decisión predicen un valor continuo, que se encuentra dentro de un rango amplio de valores, como el coste de un incidente en euros. Cuando un nuevo ejemplo cae en un nodo, el valor predicho para ese nodo se calcula como la media de todas las variables de salida (etiquetas) de los ejemplos de entrenamiento que se encuentran en ese nodo. Por ejemplo, si en un nodo hay 10 incidentes con costes de 1500, 2500, 3100, etc., la predicción para ese nodo sería la suma de todos esos costes dividida por 10.

Árboles de decisión: Regresión y Función de Coste

- Para la función de costo en la regresión, no tiene sentido usar la impureza de Gini. En su lugar, se utiliza el error cuadrático medio (Mean Squared Error - MSE). La función de coste para una división se calcula como:

$$J(K, TdK) = (M_{izquierda} / M_{total}) * MSE_{izquierda} + (M_{derecha} / M_{total}) * MSE_{derecha}.$$

- El MSE para un nodo específico se calcula como el sumatorio de las diferencias al cuadrado entre la predicción del nodo y la etiqueta real de cada ejemplo en ese nodo, dividido por el número de ejemplos en el nodo.

Limitaciones de los árboles de decisión

Los árboles de decisión son muy sensibles al overfitting (varios óptimos locales) y se mejoran con los **Random Forest** (Conjunto aleatorio de Árboles):

- Sobreajuste: Los árboles de decisión pueden crear modelos complejos que sobreajustan los datos de entrenamiento, lo que resulta en una generalización deficiente.
- Óptimos locales: El enfoque voraz del algoritmo, que optimiza las divisiones localmente, no garantiza un árbol globalmente óptimo.
- Inestabilidad: Pequeños cambios en los datos de entrenamiento pueden resultar en árboles significativamente diferentes.

Ensemble Learning: Introducción

- El Aprendizaje Conjunto (Ensemble), es una técnica diseñada para superar las limitaciones de los árboles de decisión individuales, como el sobreajuste y la inestabilidad.
- El Aprendizaje Conjunto combina predicciones de múltiples algoritmos, buscando una mejor predicción general que la de cualquier modelo individual.
- Un método clave que se analiza son los "clasificadores de votación", donde se elige la predicción más frecuente entre varios modelos.
- Se destaca Random Forest como una técnica específica de Aprendizaje Conjunto basada en el "bagging".

Ensemble Learning: Introducción

- El principio fundamental que sustenta las técnicas de Ensemble Learning es que al agregar la predicción de un conjunto de algoritmos, es muy probable que se obtenga una predicción mejor que la de un algoritmo de manera individual.
- Los métodos principales de Ensemble Learning son: bagging, pasting, boosting y stacking.

Ensemble Learning: Bagging y Pasting

- Ambos métodos agrupan múltiples instancias del mismo algoritmo de predicción, entrenando cada una con diferentes subconjuntos de datos, donde se pueden repetir los datos (traslapado).
- La diferencia clave radica en el muestreo de datos: Bagging permite repetir ejemplos dentro del subconjunto de una sola instancia de un algoritmo, mientras que Pasting no.
- Las predicciones se agregan mediante votación mayoritaria (clasificación) o promedio (regresión).

Ensemble Learning: Bagging y Pasting

- La diferencia fundamental entre Bagging y Pasting radica en cómo se manejan los ejemplos de entrenamiento dentro de los subconjuntos de datos utilizados para entrenar cada instancia del algoritmo.
- En la técnica de Bagging, un mismo ejemplo de entrenamiento puede ser seleccionado varias veces para la misma instancia del algoritmo de predicción. Esto significa que, al crear un subconjunto de datos para entrenar un modelo específico, puede haber ejemplos repetidos dentro de ese mismo subconjunto.
- Por el contrario, en la técnica de Pasting, no puede haber ejemplos de entrenamiento repetidos en el mismo subconjunto de datos con el que se entrena una instancia del algoritmo. Sin embargo, sí pueden repetirse ejemplos de entrenamiento entre diferentes subconjuntos que se utilizan para entrenar distintas instancias del algoritmo.

Ensemble Learning: Random Forests

- Ensemble Learning es un paradigma en Machine Learning que consiste en combinar múltiples modelos para mejorar el rendimiento. Estos modelos pueden ser del mismo tipo (homogéneos) o distintos (heterogéneos). Existen diferentes técnicas dentro de ensemble, como bagging, pasting, boosting y stacking.
- Random Forest es un método de ensemble específico basado en bagging, donde se entrenan múltiples árboles de decisión sobre subconjuntos de datos generados con reemplazo. Además, introduce aleatoriedad adicional al seleccionar un subconjunto de características en cada división del árbol, lo que reduce la correlación entre los árboles y mejora la capacidad de generalización del modelo.

- Random Forest construye un conjunto de árboles de decisión mediante bagging (agregación bootstrap) y un subconjunto aleatorio de características en cada división.
- Los hiperparámetros clave incluyen `n\_estimators` (número de árboles) y `max\_depth` (profundidad del árbol), que influyen en el sobreajuste, el subajuste y los recursos computacionales.
- El algoritmo se puede utilizar tanto para clasificación (voto mayoritario) como para regresión (promedio de predicciones), y ayuda a medir la importancia de las características (significativas).

- Random Forest se diferencia de un árbol de decisión estándar en que no evalúa todas las características en cada división. En su lugar, para cada nodo del árbol selecciona aleatoriamente un subconjunto de características y elige la mejor dentro de ese subconjunto. Esto introduce variabilidad entre los árboles y reduce la correlación entre ellos, mejorando la capacidad de generalización del modelo.

Random Forest: Clasificación y Regresión

En un Random Forest, las predicciones se agregan de manera diferente para tareas de clasificación y regresión:

- Para tareas de clasificación: Cuando se entrena un Random Forest con múltiples árboles de decisión, cada árbol predice una clase para un nuevo ejemplo. La predicción final del Random Forest se determina seleccionando la clase que más se repite entre las predicciones de todos los árboles. Si hay un empate en el número de predicciones para dos clases, se elegiría una de las dos clases al azar.
- Para tareas de regresión: En el caso de problemas de regresión, cada árbol de decisión en el Random Forest predice un valor continuo. Para obtener la predicción final del Random Forest, se calcula la media de todos los valores continuos predichos por cada uno de los árboles.

- Random Forest con Python
- Árboles de decisión con Python: regresión y clasificación

Sección 11: Selección y Extracción de características

Selección del Modelo: Cross-Validation (Validación cruzada)

- CV: **Cross-Validation** (Validación cruzada)
- CV = evaluar el modelo varias veces con diferentes divisiones de los datos. Si se define:
`RandomizedSearchCV(modelo, param_distributions, cv=5)`

Significa:

- Se divide el dataset en **5 partes (folds)**
- Se entrena 5 veces por cada combinación de hiperparámetros:
 - 4 partes para entrenamiento
 - 1 parte para validación
- Se rota la partición de validación en cada iteración

Evita:

- Evaluaciones sesgadas por una sola partición
- Sobreajuste al conjunto de entrenamiento

Y permite:

- Elegir hiperparámetros más confiables

Selección del Modelo: Cross-Validation

Etapa 1: Separación inicial

- train_set
- test_set

Etapa 2: Selección de modelo

- Se usa solo train_set
- Se aplica Cross-Validation

Etapa 3: Modelo final

- Se eligen los mejores hiperparámetros
- Se reentrena con todo el train_set

Etapa 4: Evaluación final

- Se usa test_set

Selección de características

- **Objetivo:** Reducir la dimensionalidad de los datos para mejorar el rendimiento del algoritmo, la calidad de los datos y la interpretabilidad del modelo.
- Para ello, se identifican características relevantes evaluando su impacto en el rendimiento del clasificador, midiendo su correlación con la variable de salida o utilizando algoritmos como árboles de decisión y conjuntos de árboles aleatorios.
- Los métodos incluyen:
 - eliminación de características con valores faltantes
 - muchos valores únicos
 - alta correlación entre las características de entrada
 - poca importancia para el modelo

Selección de características: Criterios de relevancia

- **Impacto en el rendimiento del clasificador:** Se entrena un clasificador con un conjunto de datos original. Luego, se elimina una característica y se vuelve a entrenar el clasificador con el subconjunto de datos resultante. Si el rendimiento del clasificador disminuye significativamente después de eliminar la característica, ésta se considera relevante. Si el rendimiento no se deteriora, la característica no es relevante.
- **Correlación con la característica de salida:** Se mide la correlación entre las características de entrada y la característica de salida. Cuanta más correlación haya entre una característica de entrada y la característica de salida, mayor será su relevancia. Si no hay correlación, la relevancia se considera baja.
- **Uso de algoritmos que establecen la importancia de las características:** Ciertos algoritmos, como los árboles de decisión, establecen la importancia de las características durante la construcción de su modelo. Las características que se utilizan para realizar los cortes en las diferentes ramas del árbol (es decir, para dividir el conjunto de datos original y sus subconjuntos) se consideran relevantes. El algoritmo Random Forest es un ejemplo de modelo de aprendizaje automático utilizado para identificar características con cero importancias, midiendo cómo una característica reduce la impureza de los nodos en un árbol de decisión.

Extracción de características vs Selección de Características

- Las técnicas de extracción de características se diferencian de las técnicas de selección de características en la reducción de dimensionalidad en que, mientras que la selección de características elimina algunas columnas o atributos de entrada manteniendo el resto inmutables, las técnicas de extracción de características transforman los datos de tal manera que las características de entrada resultantes son completamente diferentes de las originales.
- Con la extracción de características, el conjunto de datos resultante tendrá menos dimensiones, pero las características de entrada que queden serán nuevas y distintas de las que existían en el conjunto de datos original. Esta situación no ocurre en la selección de características.
- Observación: La extracción de características es una buena herramienta para esconder los nombres de las características originales, por ejemplo, en el caso de analizar datos bancarios, donde es importante mantener la reserva de la información de los clientes.

Extracción de características: Algoritmos claves

- Los algoritmos claves que se analizan son el PCA (Análisis de Componentes Principales) y la SVD (Descomposición en Valores Singulares).
- El PCA utiliza la SVD para proyectar datos de alta dimensión en un espacio de menor dimensión mediante la identificación de componentes principales que maximizan la varianza.
- Ver carpeta H:\Mi unidad\Carlos2026\TopicosD-IngCiv-ML-2026\materialPresentacion\S11

Sección 12: Algoritmos de Clustering

Algoritmos de Clustering: ¿Qué es?

- Los algoritmos de agrupación en clústeres agrupan datos sin etiquetar según métricas de proximidad.
- A diferencia del aprendizaje supervisado, no requieren etiquetas de salida.
- El objetivo es estructurar los datos, lo que permite la identificación de patrones o categorías.

Algoritmos de Clustering

- Los algoritmos de clustering se basan en el aprendizaje no supervisado. Esto implica que reciben un conjunto de datos sin etiquetar, es decir, un conjunto de datos de entrenamiento que solo contiene variables de entrada (como X_1 , X_2), pero carece de una variable de salida o etiqueta. El analista no proporciona etiquetas para los ejemplos del conjunto de datos.
- La finalidad de los algoritmos de clustering es proporcionar estructura a un conjunto de datos sin etiquetar, realizando agrupaciones coherentes de los ejemplos basándose en criterios matemáticos como la proximidad, la densidad o la conectividad. Estas agrupaciones se denominan clústeres.
- La métrica utilizada por los algoritmos de clustering para evaluar la cercanía entre ejemplos puede variar. Por ejemplo, en el algoritmo K-means se utiliza típicamente la distancia euclídea, buscando minimizar la suma de las distancias cuadráticas entre los datos y el centroide del clúster. Sin embargo, existen otros algoritmos que emplean diferentes conceptos de proximidad o métricas para evaluar la similitud entre los ejemplos.
- Los algoritmos no supervisados pueden utilizarse como herramientas exploratorias para descubrir patrones ocultos en grandes volúmenes de datos y, en algunos casos, como apoyo en procesos de etiquetado semi-supervisado.

Técnicas de clustering: KMEANS

K-Means es una técnica de aprendizaje no supervisado popular, sencilla y escalable para agrupar datos sin etiquetar. Funciona mediante:

- Inicialización de centroides de conglomerados aleatorios.
- Asignación de puntos de datos al centroide más cercano.
- Recalculación de las posiciones de los centroides como la media de los puntos asignados.
- Repetición hasta que los centroides se estabilicen.
- Las limitaciones incluyen:
 - La necesidad de conocer el número de conglomerados de antemano,
 - La sensibilidad al escalado de características y
 - La suposición de conglomerados esféricos, lo que lo hace inadecuado para distribuciones de datos no esféricas.

KMEANS: Funcionamiento

1. Inicialización aleatoria de centroides: El algoritmo comienza inicializando un número predefinido de "centroides" (puntos representados por estrellas, por ejemplo, una azul y una amarilla) de manera aleatoria dentro del conjunto de datos. Estos centroides se identifican con una notación como μ_1 y μ_2 para el centroide 1 y el centroide 2, respectivamente.
2. Asignación de ejemplos a clústeres: Una vez inicializados los centroides, el algoritmo realiza una primera agrupación de los datos del conjunto. Cada ejemplo del conjunto de datos sin etiquetar se asigna al clúster cuyo centroide está más cercano. La métrica utilizada para calcular esta proximidad o distancia es la distancia euclidiana. Si la distancia euclidiana de un ejemplo al primer centroide es menor que la distancia al segundo centroide, ese ejemplo se asigna al primer clúster. Cada centroide forma un clúster, por lo que el número de clústeres es igual al número de centroides.
3. Movimiento de centroides: Después de asignar todos los ejemplos a un clúster, los centroides se mueven. Cada centroide se reubica al punto medio (la media aritmética) de todos los ejemplos que se han asignado a su clúster correspondiente. Por ejemplo, para el primer centroide, su nuevo valor se calcula como la suma de todos los ejemplos en su clúster dividida por el número de ejemplos en ese clúster.
4. Iteración hasta la convergencia: Los pasos 2 y 3 se repiten en un bucle. En cada iteración, los ejemplos se reasignan a los clústeres según la nueva posición de los centroides, y luego los centroides se mueven nuevamente a la media de sus clústeres actualizados. Este proceso continúa hasta que la posición de los centroides ya no varía significativamente. Cuando los centroides dejan de moverse, significa que K-means ha terminado de construir su modelo, y los ejemplos asociados a cada centroide forman las agrupaciones o subconjuntos finales.

La función de coste que K-means trata de minimizar es la suma de las distancias euclidianas al cuadrado entre cada ejemplo de entrenamiento y el centroide de su clúster asignado.

KMEANS: Limitaciones

- **Necesidad de intuir el número de clústeres de antemano:** K-means requiere que se le proporcione el número de centroides (o clústeres) por adelantado. Si se trabaja con datos sin etiquetar, no se sabe a priori cuántas categorías existen. Una referencia útil es elegir un número de clústeres entre una y tres veces el número de etiquetas esperadas. **Este número de clústeres es un hiperparámetro que se pasa como argumento a la función de K-means.**
- **Requisito de normalización de datos:** K-means se basa en la distancia euclidiana. Si las características de entrada tienen rangos de valores muy diferentes (por ejemplo, una característica entre 0 y 2 y otra entre 0 y 100), el algoritmo priorizará las características con mayores diferencias de valores al calcular las distancias, restando importancia a las características con valores más pequeños. Por lo tanto, es crucial aplicar normalización o alguna técnica de escalado (como estandarización) para que todas las características de entrada y sus valores se encuentren en rangos similares.
- **No apto para datos categóricos con One Hot Encoding:** K-means no debe utilizarse con datos categóricos a los que se les ha aplicado One Hot Encoding. Si se codifican valores categóricos (como "azul", "amarillo", "rojo") como 0, 1 y 2, el algoritmo interpretará que hay una relación de distancia entre ellos (por ejemplo, que el azul está más cerca del amarillo que del rojo), lo cual no tiene sentido para categorías nominales. En su lugar, **se deben codificar estas características como "multiple binary".**
- **Pérdida de eficiencia con muchas dimensiones:** K-means pierde eficiencia y genera peores clústeres en conjuntos de datos con un número muy elevado de dimensiones (muchas características de entrada). Para mitigar esto, es una práctica común reducir las dimensiones del conjunto de datos mediante técnicas como PCA (Análisis de Componentes Principales) o SVD (Descomposición en Valores Singulares).
- **Sensibilidad a la inicialización de centroides:** Aunque funciona mejor si los centroides iniciales se eligen aleatoriamente, esta aleatoriedad puede provocar que los resultados cambien dependiendo de dónde se inicialicen. Una reinicialización puede llevar a la formación de clústeres diferentes.
- **Asunción de clústeres esféricos:** K-means asume que los clústeres son esféricos y no funciona correctamente en distribuciones de datos no esféricas. Si los datos tienen formas complejas o anulares, K-means no será capaz de identificar correctamente la estructura subyacente, ya que su mecanismo de asignación de puntos al centroide más cercano tiende a crear límites de decisión lineales o hiperplanos, resultando en clústeres de forma esférica o convexa.

Técnicas de clustering: DBSCAN

- DBSCAN gestiona agrupaciones no esféricas e infiere automáticamente su número.
- Los parámetros clave son ϵ (radio) y min_points (vecinos mínimos).
- Las limitaciones incluyen la sensibilidad a las variaciones en la densidad de agrupaciones y la alta dimensionalidad.
- DBSCAN no se basa en la distancia como KMEANS, sino en un concepto de densidad. Busca zonas de alta densidad de ejemplos para formar clusters, en lugar de la distancia entre ellos.

DBSCAN: Funcionamiento

DBSCAN define e identifica los puntos de la siguiente manera:

- **Core points** (Puntos centrales): Un ejemplo se considera un "core point" si tiene al menos un número mínimo de ejemplos (definido por el parámetro `min points`) dentro de su radio epsilon. Es decir, si al calcular el radio epsilon alrededor de un punto, éste contiene `min points` o más vecinos, se clasifica como un "core point".
- **Border points** (Puntos frontera): Un ejemplo se clasifica como "border point" si no es un "core point", pero se encuentra dentro del radio epsilon de un "core point". Esto significa que, aunque no tiene suficientes vecinos para ser un "core point" por sí mismo, está lo suficientemente cerca de un "core point" como para ser parte de su clúster.
- **Noise points** (Puntos de ruido): Son aquellos ejemplos que no son ni "core points" ni "border points". Esto ocurre cuando un punto no tiene suficientes vecinos dentro de su radio epsilon para ser un "core point", y tampoco se encuentra dentro del radio epsilon de ningún "core point". Estos puntos se consideran anomalías y no se incluyen en ningún clúster.

DBSCAN: Limitaciones

- **No funciona correctamente con clusters de diferentes densidades:** DBSCAN no es capaz de identificar algunos clusters si las densidades de estos varían mucho. Esto se debe a que el algoritmo se basa en la existencia de suficientes ejemplos (vecinos) dentro de un rango ϵ para formar un cluster. Si la densidad de los clusters no es alta (es decir, los ejemplos están muy separados), DBSCAN no podrá identificarlos.
- **Dependencia de la selección de parámetros:** Aunque DBSCAN infiere el número de clusters automáticamente, la selección de los parámetros ϵ (radio) y min points (número mínimo de vecinos) es crucial para su correcto funcionamiento. Estos parámetros determinan completamente cómo funcionará el algoritmo, qué clusters se formarán y si se formará alguno. Es necesario tener intuiciones previas sobre el conjunto de datos para seleccionar valores adecuados para estos parámetros.
- **No se comporta adecuadamente en conjuntos de datos con muchas dimensiones:** Al igual que K-means, DBSCAN no funciona bien con conjuntos de datos de alta dimensionalidad. En estos casos, es probable que sea necesario aplicar técnicas de reducción de dimensionalidad, como la selección o extracción de características (por ejemplo, con SVD o PCA).
- **Cuidado al alterar las densidades del conjunto de datos al realizar divisiones:** Es fundamental tener precaución al dividir el conjunto de datos (por ejemplo, en entrenamiento, validación y prueba) para no alterar las densidades. Si se alteran las densidades, se modifica el funcionamiento del algoritmo, que se basa en ellas. Esto podría llevar a la formación de clusters no significativos o a la destrucción de clusters existentes en el conjunto de datos original. Para evitar esto, se recomienda mantener las densidades en cada partición o construir los conjuntos de datos de entrenamiento y prueba de manera independiente.

Evaluación de los resultados de Clustering

- **Datos Supervisados:** Se utilizan métricas como homogeneidad, completitud, medida V y pureza cuando se dispone de etiquetas de verdad de campo. La pureza se calcula asignando cada conglomerado a la clase más frecuente dentro de él y midiendo las asignaciones correctas.
- **Datos No Supervisados:** Cuando no hay etiquetas, se emplean el coeficiente de silueta (silhouette) (que mide la cohesión intraconglomerado y la separación entre conglomerados) y el índice de Kalinsky-Harabasz (que evalúa la separación y densidad de conglomerados). El coeficiente de silueta varía de -1 a 1, y los valores cercanos a 1 indican una mejor agrupación. El índice de Kalinsky-Harabasz es más alto para conglomerados más densos y separados.

Métricas para evaluar Datasets etiquetados

Aunque los algoritmos de clustering son de aprendizaje no supervisado y no necesitan etiquetas para formar los clusters, si se dispone de ellas, se pueden utilizar para evaluar la calidad de la agrupación. En este escenario, se le proporcionan al algoritmo solo las características de entrada, sin las etiquetas. Una vez que el algoritmo ha formado los clusters, se utilizan las etiquetas originales para comparar cómo se han agrupado los ejemplos.

Existen varias métricas para evaluar los resultados de clustering con un conjunto de datos etiquetado:

- **Homogeneidad:** Mide el grado en que los clusters contienen miembros de una clase determinada. Un cluster es perfectamente homogéneo si contiene solo miembros de una única clase.
- **Plenitud:** Mide el grado en que todos los miembros de una clase determinada han sido asignados al mismo cluster. Un cluster es perfectamente completo si todos los miembros de una clase están contenidos en él.
- **V-Measure:** Es una métrica que combina la homogeneidad y la plenitud, similar al F1 Score en clasificación, y representa la precisión de las operaciones de agrupamiento.
- **Pureza:** Para calcularla, cada grupo se asigna a la clase más frecuente dentro de ese grupo. Luego, la precisión de esta asignación se mide contando el número de documentos asignados correctamente y dividiéndolo por el número total de elementos. Un ejemplo de cálculo de pureza implica sumar el número de elementos de la clase mayoritaria en cada cluster y dividirlo por el número total de ejemplos.

Métricas para evaluar Datasets no etiquetados

- Las métricas que requieren etiquetas (como homogeneidad, plenitud y pureza) no pueden utilizarse.
- **Coeficiente de Silhouette:** Se calcula de manera independiente para cada ejemplo del conjunto de datos, utilizando una métrica de distancia (por ejemplo, la distancia euclídea). Involucra dos distancias para cada muestra X: Distancia A: La distancia media entre la muestra X y todas las demás muestras en el mismo cluster. Se busca que esta distancia sea lo más pequeña posible. Distancia B: La distancia media entre la muestra X y todas las demás muestras en el siguiente cluster más cercano. Se busca que esta distancia sea lo más grande posible.
- La fórmula del coeficiente de Silhouette es $(B - A) / \max(A, B)$. El resultado se encuentra siempre entre -1 y 1. Cuanto más cercano a 1, mejor se considera el resultado del algoritmo de clustering (clusters bien separados y densos internamente). Un resultado cercano a 0 significa que los clusters se superponen. Un resultado cercano a -1 indica una mala agrupación, donde hay más distancia entre un elemento y los de su propio cluster que con los de otro cluster. Funciona bien para algoritmos de clustering basados en distancia como K-means, pero no para algoritmos basados en densidad como DB-SCAN.
- **Índice de Kalinsky (Calinski-Harabasz Index):** Considera que el resultado del algoritmo de clustering es bueno si se forman clusters visiblemente separados y densamente reunidos. Por ello, es útil para algoritmos basados en densidad como DBSCAN.
- Para su cálculo, utiliza dos medidas: WK (dispersión dentro del cluster): Una matriz de distancias entre las muestras de un cluster y el centro geométrico del mismo. Se busca que esta dispersión sea lo menor posible. BK (dispersión entre grupos): Una matriz de distancias entre el centro de un cluster y los centros de todos los clusters restantes. Se busca que esta dispersión sea lo mayor posible. Cuanto mayor sea el resultado de esta métrica, más densos y separados estarán los clusters, indicando una mejor agrupación. A diferencia del coeficiente de Silhouette, el índice de Kalinsky no proporciona un resultado comprendido dentro de un rango fijo, lo que puede dificultar su interpretación directa sin un valor de referencia.